

AGENTS & AGENTIC AI

From Zero to Production + Interview Ready

Master guide for SDE, AI Engineer, ML Engineer and Generative AI interviews.

How to Study This Guide

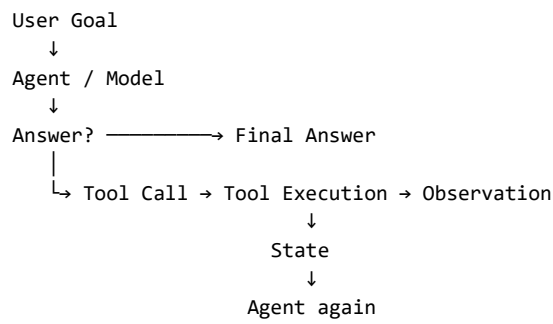
- Beginner → understand the concepts and diagrams.
- Intermediate → implement tools and the agent loop.
- Advanced → planning, memory, ReAct, multi-agent and LangGraph.
- Production → security, evaluation, cost, latency and reliability.
- Interview → explain trade-offs and implement a basic agent without a framework.

PART 1 — WHAT IS AN AI AGENT?

1. What is an AI Agent?

An AI agent is a software system where an AI model can decide what action to take, invoke available tools, observe results, update its state and continue until a goal is completed or a safe stopping condition is reached.

Hinglish intuition: LLM ko smart employee samjho. 'Python kya hai?' par woh answer dega. 'Meri meetings check karo, conflict find karo, aur mujhe batao' ke liye usse calendar tool use karna, result dekhna aur next action decide karna padega. Ye agentic behavior hai.



Why do we need agents?

- LLMs generate information but normally cannot directly perform real-world actions.
- Some tasks require multiple steps and the next step depends on the previous result.
- Agents can dynamically select among APIs, databases, search, retrieval, code execution and other tools.
- Agents can adapt instead of following only one fixed pipeline.

Real-world examples

Use case	Possible agent actions
Travel assistant	Search flights → compare → check dates → build itinerary
Coding agent	Inspect repo → search code → edit → run tests → fix
Support agent	Find customer → retrieve policy → calculate eligibility → respond
Research agent	Search → extract → compare → verify → synthesize

When NOT to use an agent

- A deterministic function already solves the task.
- The process is always $A \rightarrow B \rightarrow C$.
- You need extremely predictable latency/cost.
- Autonomy adds risk without useful decision-making.

Interview-ready: An AI agent combines a model with tools, state and a control loop so it can select actions, observe results and iterate toward a goal.

PART 2 — AGENT vs LLM vs RAG

System	Core idea	Actions	Knowledge
LLM	Generate text	Usually none	Training + prompt
RAG	Retrieve evidence then generate	Usually no external side effects	External documents
Agent	Decide + act + observe + iterate	Yes, through tools	Can use RAG/tools/APIs

LLM: Query → Model → Answer
RAG: Query → Retrieve → Context → Model → Answer
Agent: Goal → Model → Tool → Observation → Model → Tool/... → Answer

RAG can be a tool inside an agent. Agentic RAG means the agent decides when, where and how to retrieve, potentially repeating retrieval or combining retrieval with other tools.

PART 3 — TOOLS

What is a tool?

A tool is an executable capability exposed to an agent through a defined interface. The model proposes a tool call; trusted application code validates and executes it.

Tool	Input	Output
Calculator	Expression	Number
Web search	Query	Results
Vector search	Query + filters	Documents
SQL	Structured request	Rows
Calendar	Date range	Events
Code runner	Code	Output/errors

```
name: get_weather
description: Get current weather for a city.
schema:
{
  "city": "string",
  "units": "string"
}
```

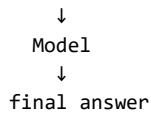
Tool design rules

- Clear name and description.
- Strict schema.
- Server-side validation.
- Least privilege.
- Timeout and retry policy.
- Structured output.
- Explicit side-effect behavior.
- Useful error messages.

PART 4 — FUNCTION / TOOL CALLING

Function/tool calling allows a model to return a structured request such as a function name plus arguments. Your application then executes the real function and returns the result to the model.

```
User → Model
      ↓
  tool_call:
    name = get_weather
    args = {"city": "Delhi"}
      ↓
  validate + authorize
      ↓
  execute tool
      ↓
  tool result
```



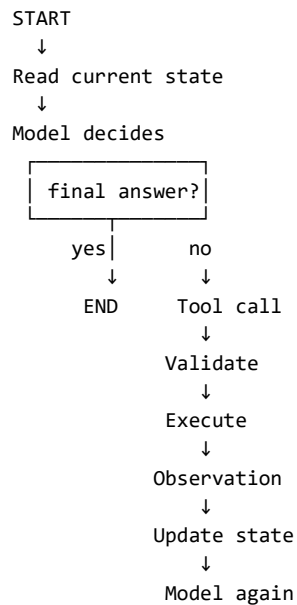
Critical interview point: the model does NOT execute your Python function or API. It requests a tool call. The application/runtime performs execution.

```

while True:
    response = model(messages, tools=tools)

    if response.has_tool_call():
        for call in response.tool_calls:
            args = validate(call.arguments)
            result = execute(call.name, args)
            messages.append(response)
            messages.append(tool_result(call.id, result))
    else:
        return response.text
  
```

PART 5 — THE AGENT LOOP



Step-by-step

1. Receive goal.
2. Load instructions and current state.
3. Expose allowed tools.
4. Ask model for next action.
5. Validate the requested action.
6. Execute the tool.
7. Store observation.
8. Update state.
9. Repeat.
10. Stop on success, failure, timeout, approval requirement or hard budget.

Why stopping conditions?

- Maximum steps

- Maximum cost/tokens
- Timeout
- Repeated tool-call detection
- Tool failure limit
- Explicit success condition
- Human approval checkpoint

Without limits, an agent can loop, repeatedly call an API, consume tokens or perform repeated side effects.

PART 6 — PLANNING

Planning means choosing actions that can achieve a goal. It can be explicit (a task list) or implicit (choose only the next action).

Goal: Compare two laptops under ₹80,000

- Plan:
1. Search specs for A
 2. Search specs for B
 3. Get current prices
 4. Compare CPU/RAM/battery
 5. Check budget
 6. Recommend with sources

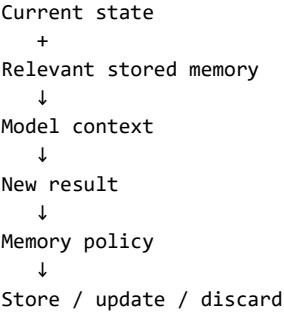
Strategy	Idea	Trade-off
Fixed plan	All steps generated upfront	Simple but brittle
Reactive	Choose next action from current state	Flexible but less predictable
Plan + execute	Planner creates tasks, executor performs them	Good for complex tasks
Hierarchical	Goal → subgoals → tasks	Powerful but more overhead

Replanning matters because the world can invalidate the original plan. A good agent treats a plan as a hypothesis, not a guarantee.

PART 7 — MEMORY

Memory is information stored by the system that can be selected into future context. Context is what the model sees in one particular call; memory is persistent information that may be retrieved later.

Memory	Purpose	Example
Working state	Current execution	Current tool result
Conversation	Prior dialogue	Previous question
Semantic long-term	Durable facts	User prefers Python
Episodic	Past events	Previous successful workflow
Procedural	How to do something	Deployment procedure
External memory	DB/vector store	Past tickets



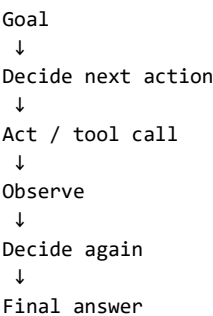
Memory risks

- Stale facts
- Privacy leakage
- Wrong-user association
- Memory poisoning
- Unbounded growth
- Irrelevant retrieval

Production rule: do not store everything. Define what may be remembered, why, for how long, who can access it, and how it can be deleted.

PART 8 — ReAct

ReAct is a reasoning-and-acting pattern in which the system alternates between deciding what to do, taking an action and observing the result. In modern systems, the important engineering idea is the action-observation loop; internal chain-of-thought should not be treated as something an application must expose.



Question: What is the weather in the city of my next meeting?

Action: `calendar.get_next_meeting()`
Observation: Chandigarh

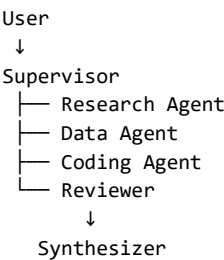
Action: `weather.get_current("Chandigarh")`
Observation: 27°C, clear

Final: Next meeting is in Chandigarh; current weather is 27°C and clear.

ReAct is a control pattern, not a synonym for every possible agent architecture.

PART 9 — MULTI-AGENT SYSTEMS

A multi-agent system uses multiple specialized agents that collaborate, delegate, review or execute different parts of a task.



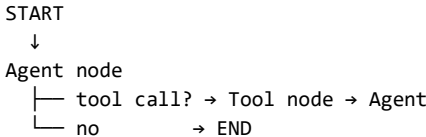
Pattern	Flow	Good for
Supervisor	Manager → specialists → manager	Complex workflows

Sequential	A → B → C	Fixed stages
Parallel	A/B/C → aggregator	Independent tasks
Debate/review	Draft → critique → improve	Quality checks
Hierarchical	Manager → sub-managers → workers	Large tasks

Do not use multi-agent systems merely because they sound advanced. They add coordination, token cost, latency and failure modes.

PART 10 — LANGGRAPH

LangGraph is a framework for building stateful, graph-based agent workflows. The central mental model is nodes + edges + shared state + conditional loops.



State

State is structured execution information: messages, tool results, retrieved documents, plan, counters, approvals and status.

Node

A node performs one unit of work: model call, retrieval, tool execution, validation, reviewer or human approval.

Edge

An edge controls what runs next. Conditional edges allow routing based on current state.

```

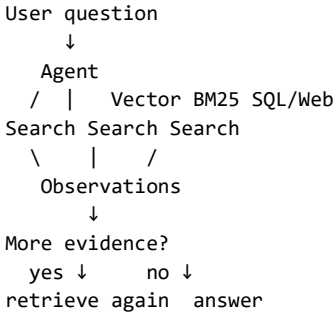
class State(TypedDict):
    messages: list
    steps: int

START → agent
agent → tool    if tool call exists
agent → END    otherwise
tool  → agent
  
```

For interviews, explain the graph and state model first. Framework APIs change; the architecture concepts remain.

PART 11 — AGENTIC RAG

Agentic RAG combines retrieval with an agent control loop. The agent can decide which source to use, reformulate a query, retrieve again, combine evidence or use another tool.



Normal RAG	Agentic RAG
Fixed retrieval flow	Dynamic retrieval decisions
Usually one retrieval stage	Can retrieve multiple times
Predictable	Adaptive
Lower complexity	Higher complexity
Great for direct QA	Great for multi-hop research

Example

Question: 'Compare our 2025 and 2026 leave policies and explain what changed for contractors.'

- 11. Retrieve 2025 policy.
- 12. Retrieve 2026 policy.
- 13. Retrieve contractor-specific sections.
- 14. Compare evidence.
- 15. If evidence is incomplete, perform targeted retrieval.
- 16. Generate a cited answer.

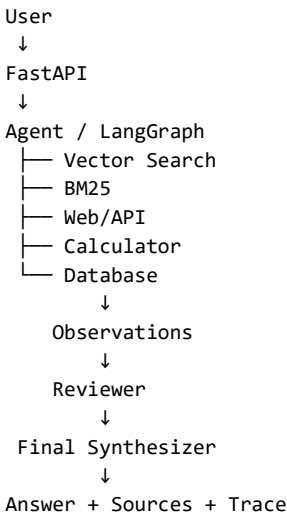
When to use

- Multi-hop questions
- Multiple knowledge sources
- Iterative research
- Dynamic retrieval choice
- Questions combining documents with APIs/SQL

When not to use

- Simple FAQ
- Straightforward document QA
- Strict latency requirements
- No safe tool authorization available

PART 12 — END-TO-END PROJECT: INTELLIGENT RESEARCH AGENT



Layer	Suggested technology	Reason
API	FastAPI	Clear Python service boundary
Orchestration	LangGraph or custom loop	Explicit state/control
Retrieval	pgvector/FAISS/Qdrant	Vector search

Lexical	BM25	Exact terminology
Embeddings	Hugging Face/hosted	Semantic search
LLM	Local/API model	Generation + decisions
State	PostgreSQL/Redis	Persistence depending on need
Frontend	React	Interactive UI
Deployment	Docker	Reproducibility

Folder structure

```

agentic-research/
├── app/
│   ├── main.py
│   ├── agent/
│   │   ├── graph.py
│   │   ├── state.py
│   │   ├── prompts.py
│   │   └── policies.py
│   ├── tools/
│   │   ├── search.py
│   │   ├── web.py
│   │   ├── calculator.py
│   │   └── database.py
│   ├── retrieval/
│   │   ├── embeddings.py
│   │   ├── vector_store.py
│   │   ├── bm25.py
│   │   └── reranker.py
│   ├── memory/
│   │   └── store.py
│   └── evaluation/
├── tests/
├── Dockerfile
└── README.md

```

Example state

```

state = {
    "user_query": "...",
    "messages": [],
    "plan": [],
    "tool_results": [],
    "retrieved_docs": [],
    "citations": [],
    "step_count": 0,
    "status": "running"
}

```

Safety policy

- Read-only by default.
- Validate every tool call.
- Per-tool authorization.
- Max steps/time/cost.
- Explicit approval for risky actions.
- Never let retrieved text override system policy.
- Trace tool calls and outcomes without leaking secrets.

PART 13 — AGENT EVALUATION

Agents can fail at tool choice, arguments, planning, execution, state management, stopping or final generation. Therefore final-answer accuracy alone is insufficient.

Metric	Question
Task success	Did the task finish correctly?
Tool selection accuracy	Was the right tool chosen?
Argument accuracy	Were parameters correct?
Trajectory quality	Were steps useful or wasteful?
Groundedness	Is the answer supported?
Latency	How long did it take?
Cost	How much model/tool compute was used?
Safety violations	Did it attempt unauthorized actions?

```
Run
├─ Goal
├─ Model decision
├─ Tool + arguments
├─ Tool result
├─ State update
├─ Next decision
└─ Final answer
```

Trace-based debugging lets you identify whether the problem was the model decision, tool, retrieval, state update or final synthesis.

PART 14 — PRODUCTION AGENTS

Area	Risk	Control
Reliability	Tool/model failure	Retries, fallback, timeout
Cost	Long loops	Step/token budgets
Latency	Sequential calls	Parallel independent work
Security	Powerful tools	Least privilege + authorization
State	Corruption/staleness	Typed state + validation
Observability	Hard debugging	Tracing + metrics
Side effects	Duplicate actions	Idempotency
High impact	Unsafe autonomous action	Human approval
Injection	Untrusted content	Policy outside model
Multi-tenancy	Cross-user leakage	Tenant-scoped state/tools

Idempotency

If an agent retries 'create order' after a timeout, two orders could be created. An idempotency key lets the service recognize a repeated request and preserve one intended side effect.

Human-in-the-loop

For payments, deletion, publishing, external messages or other high-impact actions, separate proposal from execution and require approval when appropriate.

PART 15 — AGENT FAILURE MODES

Failure	Cause	Solution
Infinite loop	No stop condition	Max steps + duplicate detection
Wrong tool	Ambiguous tools	Better descriptions + routing
Bad arguments	Weak validation	Strict schemas + server validation

Tool hallucination	Invented capability	Only registered tools
Planning failure	Bad assumptions	Replanning + checkpoints
Stale memory	Old information	TTL/versioning
Memory poisoning	Untrusted writes	Memory policy + provenance
Prompt injection	External text controls agent	Treat content as untrusted
Over-agenting	Agent used unnecessarily	Prefer deterministic workflow
Under-agenting	Fixed flow cannot branch	Controlled agentic step
Duplicate side effect	Retry repeats action	Idempotency
Context overflow	Too much history	Prune/summarize/retrieve
Wrong stopping	Stops early/late	Explicit success + limits
Multi-agent deadlock	Agents hand off forever	Supervisor/handoff limits

PART 16 — AGENT SECURITY

Threat	Example	Defense
Prompt injection	Web page says 'ignore policy'	Separate data from instructions
Excessive agency	Agent can delete production data	Least privilege
Tool abuse	Payment tool called incorrectly	Validation + approval
Data leakage	Secret appears in tool output	Redaction + access control
Memory poisoning	Malicious fact stored	Trusted-source/write policy
Cross-tenant leakage	User A sees User B state	Tenant authorization
SSRF/tool abuse	Agent reaches internal endpoints	Network egress controls

Most important principle: the model is not a security boundary. Authorization, validation, network controls and business policies must live in trusted software.

PART 17 — AGENT ARCHITECTURE PATTERNS

Single agent

User → Agent → Tools → Agent → Answer

Router

```

User → Router
      |
      ├── SQL agent
      ├── RAG agent
      └── Coding agent
  
```

Supervisor

```

Supervisor
  ├── Researcher
  ├── Analyst
  └── Reviewer
      ↓
    Final
  
```

Planner-executor

Goal → Planner → Tasks → Executor(s) → Results → Synthesizer

Reflection

Draft → Reviewer → Feedback → Agent → Improved Draft

Human approval

Agent proposes → policy check → human approval → execute

PART 18 — AGENTS WITHOUT FRAMEWORKS

For interviews, implement the basic loop yourself. Frameworks mainly add state management, graphs, persistence, streaming, integrations and observability.

```
def run_agent(goal, model, tools, max_steps=8):
    messages = [{"role": "user", "content": goal}]

    for _ in range(max_steps):
        response = model(messages, tools=tools)

        if not response.tool_calls:
            return response.text

        messages.append(response)

        for call in response.tool_calls:
            args = validate(call.name, call.arguments)
            result = execute(call.name, args)
            messages.append(
                tool_result(call.id, result)
            )

    return "Stopped safely: step limit reached."
```

Interview point: LangGraph/LangChain does not replace understanding of APIs, state, tool execution, permissions, retrieval and evaluation.

PART 19 — WORKFLOW vs AGENT

Workflow	Agent
Developer defines sequence	Model can select sequence
Predictable	Adaptive
Easier to test	Harder to test
Lower variance	Higher variance
Good for stable processes	Good for uncertain tasks
Usually lower cost	Can require many calls

Strong production systems are often hybrids: deterministic business logic outside, bounded agentic decisions inside.

PART 20 — ADVANCED CONCEPTS

Tool routing

Select or expose only relevant tools for the current task. Fewer tools improve selection accuracy and reduce attack surface.

Structured output

Constrain model responses to a schema such as action, arguments, status or next state. Validate before execution.

Guardrails

Trusted controls around inputs, actions, outputs and state: schemas, policy checks, permissions, limits and approvals.

Checkpointing

Persist execution state so an agent can recover, pause for approval or resume after restart.

Parallel tool calls

Independent searches can run concurrently to reduce latency. Do not parallelize dependent operations.

Compensation

If a later step fails, a compensating action can reverse an earlier side effect, such as releasing inventory after payment failure.

PART 21 — COMPLETE COMPARISON

Approach	Main idea	Strength	Weakness	Use case
LLM	Generate	Simple	No actions	Text tasks
RAG	Retrieve + generate	Grounded knowledge	Fixed retrieval	Document QA
Workflow	Fixed steps	Predictable	Less adaptive	Business process
Single Agent	Dynamic tool loop	Flexible	Variable	Assistants
ReAct	Action-observation	Simple agent pattern	More calls	Tool tasks
Planner-Executor	Plan then execute	Complex goals	Plan can fail	Research
Multi-Agent	Specialists collaborate	Specialization	Coordination cost	Large tasks
LangGraph	Stateful graph	Explicit control	Framework complexity	Production agents
Agentic RAG	Agent controls retrieval	Adaptive research	Cost/latency	Multi-hop QA

PART 22 — BEGINNER INTERVIEW QUESTIONS

Q: What is an AI agent?

Answer: A model-centered system that can select actions/tools, observe results, maintain state and iterate toward a goal.

Q: Agent vs LLM?

Answer: An LLM generates; an agent adds tools, state and a control loop.

Q: Agent vs RAG?

Answer: RAG retrieves evidence; an agent can dynamically decide retrieval and other actions.

Q: What is a tool?

Answer: An executable capability exposed through a defined interface.

Q: What is function calling?

Answer: Structured model output requesting a declared function/tool with arguments.

Q: Does the model execute the tool?

Answer: No. The trusted application executes it.

Q: What is the agent loop?

Answer: Decision → tool → observation → state update → next decision.

Q: What is state?

Answer: Current structured execution information.

Q: What is planning?

Answer: Selecting actions to reach a goal.

Q: What is memory?

Answer: Persisted information that can be retrieved into future context.

Q: What is ReAct?

Answer: Interleaving action selection and observations in a loop.

Q: What is multi-agent AI?

Answer: Multiple specialized agents coordinating.

Q: What is LangGraph?

Answer: A graph/state framework for agent workflows.

Q: What is Agentic RAG?

Answer: Agent-controlled, iterative or dynamic retrieval.

Q: When should you not use agents?

Answer: When deterministic logic is sufficient.

Q: Why tools?

Answer: To access external data/capabilities.

Q: Why schemas?

Answer: To constrain and validate arguments.

Q: What is an observation?

Answer: A tool/action result.

Q: Why max steps?

Answer: Prevent infinite loops and runaway cost.

Q: What is human-in-the-loop?

Answer: Human review/approval of selected actions.

Q: What is least privilege?

Answer: Minimum permissions required.

Q: What is checkpointing?

Answer: Persisting state for recovery/resume.

Q: What is a supervisor?

Answer: Coordinator delegating work to specialist agents.

Q: What is a router?

Answer: Chooses a suitable tool/agent/workflow.

Q: What is structured output?

Answer: Schema-constrained model output.

Q: Why agents cost more?

Answer: They may perform multiple model/tool calls.

Q: Why agents are slower?

Answer: Sequential calls add latency.

Q: What is tool validation?

Answer: Checking arguments and permissions before execution.

Q: What is prompt injection?

Answer: Untrusted text tries to influence instructions/actions.

Q: What is bounded autonomy?

Answer: Autonomy constrained by tools, permissions and hard limits.

PART 23 — INTERMEDIATE INTERVIEW QUESTIONS

Q: How do you prevent infinite loops?

Answer: Max steps, timeout, cost budget, duplicate detection and explicit success criteria.

Q: How do you design a tool?

Answer: Clear description, strict schema, validation, authorization, timeout and structured output.

Q: How do you handle tool failure?

Answer: Retry safe transient errors, use fallback when appropriate, update state and stop safely if needed.

Q: State vs memory?

Answer: State is current run information; memory persists useful information beyond the run.

Q: What should be remembered?

Answer: Useful authorized durable facts, not every message.

Q: Why ReAct?

Answer: Observations can change the next action.

Q: Plan-and-execute?

Answer: Plan tasks, execute them, synthesize and optionally replan.

Q: Why multi-agent?

Answer: Specialization, delegation and parallelism.

Q: Why can multi-agent be worse?

Answer: Coordination, latency, cost and debugging overhead.

Q: What is a LangGraph node?

Answer: A unit of work.

Q: What is an edge?

Answer: A transition between nodes.

Q: Why graph orchestration?

Answer: Explicit state, branching, loops and checkpoints.

Q: How evaluate tool selection?

Answer: Labeled tasks with correct tool and argument expectations.

Q: How reduce cost?

Answer: Limit steps, route to cheaper models, cache, parallelize and prune context.

Q: How reduce latency?

Answer: Parallelize independent calls, cache and reduce sequential model calls.

Q: What is idempotency?

Answer: Safe repetition without duplicate side effects.

Q: Why are side effects dangerous?

Answer: Retries/model mistakes can repeat real-world actions.

Q: How handle side effects?

Answer: Validation, authorization, idempotency and approval.

Q: What is a guardrail?

Answer: Trusted constraint around behavior.

Q: What is tool routing?

Answer: Selecting relevant tools.

Q: What is checkpointing?

Answer: Saving state for recovery/approval.

Q: How prevent stale memory?

Answer: TTL, timestamps and update/delete policies.

Q: How handle conflicting memory?

Answer: Use provenance/authority/recency or ask the user.

Q: How debug agents?

Answer: Trace model decisions, tool calls, results and state.

Q: Why not expose every tool?

Answer: Selection gets harder and attack surface increases.

Q: How does an agent use RAG?

Answer: RAG can be exposed as a retrieval tool.

Q: Can RAG exist without agents?

Answer: Yes.

Q: Can agents exist without RAG?

Answer: Yes.

Q: What is a supervisor?

Answer: Coordinator that delegates and aggregates.

Q: What is a reviewer agent?

Answer: Checks a result against evidence/criteria.

Q: What is a hybrid system?

Answer: Deterministic workflow plus bounded agentic components.

Q: What is an approval gate?

Answer: A pause before a risky action until trusted approval.

Q: What is tool sandboxing?

Answer: Restricting the runtime/resources accessible to a tool.

Q: Why version prompts/models?

Answer: To reproduce and debug behavior.

Q: Why trace state?

Answer: Agent failures are often multi-step and invisible from final output alone.

Q: What is a trajectory?

Answer: The sequence of decisions, actions and observations.

Q: What is over-agenting?

Answer: Using an agent where simple code is better.

Q: What is under-agenting?

Answer: A rigid pipeline unable to adapt to task uncertainty.

Q: How route SQL vs RAG?

Answer: Use task classification/routing based on whether the question is structured or unstructured.

Q: Why use structured tool outputs?

Answer: Easier validation and deterministic downstream handling.

PART 24 — ADVANCED INTERVIEW QUESTIONS

1. How would you safely design a flight-booking agent?

Read-only search first; confirm price/details; require explicit approval; validate arguments; use idempotency and audit logs for booking.

2. How prevent an agent deleting production data?

Do not expose destructive tools by default; use least privilege, policy gates and approval.

3. How evaluate an agent beyond final accuracy?

Task success, tool selection, arguments, trajectory efficiency, latency, cost and safety.

4. How design memory?

Separate working state and durable memory; store only authorized useful facts with provenance and retention rules.

5. How handle prompt injection from web content?

Treat it as untrusted data and prevent it from changing system/tool permissions.

6. How build multi-agent research?

Supervisor delegates independent subtasks, specialists gather evidence, reviewer checks it, synthesizer produces result.

7. How prevent multi-agent loops?

Limit handoffs, enforce ownership and add supervisor termination rules.

8. When parallelize tools?

When operations are independent.

9. How handle conflicting agent outputs?

Compare evidence with a reviewer; prefer authoritative sources.

10. How choose a model?

Benchmark tool-use reliability, task performance, latency and cost on representative tasks.

11. How reduce hallucinated arguments?

Strict schemas, validation and clear descriptions.

12. How handle tool-output injection?

Treat outputs as untrusted and do not allow them to override policies.

13. How resume after crash?

Persist checkpoints and make side effects idempotent.

14. How guarantee stopping?

Use external hard limits; don't rely only on prompts.

15. How handle long context?

Summarize, prune and retrieve only relevant state.

16. How does Agentic RAG improve RAG?

It can choose sources, reformulate and iterate when evidence is insufficient.

17. Main downside of Agentic RAG?

Variable latency, cost and failure surface.

18. How route SQL vs RAG?

Classify structured numerical requests to SQL and unstructured semantic requests to retrieval.

19. How protect secrets?

Keep credentials in trusted runtime, never in prompts or model-visible instructions.

20. How audit agents?

Trace run ID, model/version, tool calls, authorization and state transitions.

21. How test agents?

Unit-test tools, integration-test graph paths, scenario-test trajectories and security-test untrusted inputs.

22. What is a bounded agent?

An agent with explicit tool, step, time, cost and permission limits.

23. Why bounded autonomy?

It turns open-ended model behavior into controllable software.

24. How design a coding agent?

Sandbox repo, read/search/edit/test tools, approval for destructive actions.

25. How design customer support?

Scoped customer APIs + policy retrieval + draft response + approval for high-impact credits/refunds.

26. How design incident response?

Read telemetry → diagnose → propose remediation → approval → execute.

27. What is compensation?

Action that reverses a previous side effect after failure.

28. Why retries are dangerous?

They may repeat side effects; use idempotency.

29. How are agents security risks?

They combine uncertain model decisions with powerful tools and untrusted inputs.

30. How control web access?

Allowlist/network controls and treat content as untrusted.

31. How measure production quality?

Task success, escalation, tool errors, latency, cost, feedback and safety events.

32. When multi-agent is justified?

When specialization/parallelism produces a measurable benefit over one agent.

33. Why use LangGraph?

Explicit state, graph routing, loops and checkpointing.

34. Why not just say 'I used LangChain'?

Framework use is not architecture; explain tools, state, retrieval, permissions and evaluation.

35. Most important agent principle?

Give useful autonomy while keeping execution and authorization under trusted control.

36. How handle no-answer?

Agent should abstain/escalate when evidence is insufficient instead of inventing.

37. How handle model upgrades?

Run regression evaluations before production rollout.

38. How handle tool versioning?

Version schemas/contracts and test old/new behavior.

39. How prevent cross-tenant tool calls?

Authorize every tool request against authenticated tenant/user context.

40. How design an agent for regulated data?

Strict data boundaries, auditability, minimal retention, authorization and human oversight.

PART 25 — SCENARIO-BASED INTERVIEW QUESTIONS

Scenario 1: Agent repeats the same search

Inspect trace; add caching, duplicate-call detection and max steps.

Scenario 2: Agent books twice after timeout

Use idempotency keys and verify transaction status before retry.

Scenario 3: Wrong tool chosen

Improve descriptions/schema, reduce exposed tools and add tool-routing evaluation.

Scenario 4: Correct tool, wrong args

Strict schema + server validation + clarification for ambiguity.

Scenario 5: Memory leaks another user's data

Scope memory by authenticated user/tenant and test authorization.

Scenario 6: Web page says ignore policy

Treat web content as untrusted data.

Scenario 7: Agent takes 20 steps for simple QA

Route simple queries to deterministic RAG/workflow.

Scenario 8: Multi-agent system too slow

Parallelize independent tasks and reduce handoffs.

Scenario 9: Agents disagree

Use evidence-based reviewer rather than blind voting.

Scenario 10: Research never finishes

Inspect planning/retrieval and add bounded replanning.

Scenario 11: Cost is too high

Track per-step usage; reduce calls/context and use routing/caching.

Scenario 12: Tool API is flaky

Timeout + safe retry + circuit breaker + fallback.

Scenario 13: Agent crashes midway

Persist checkpoints and resume safely.

Scenario 14: User requests destructive action

Require policy check and approval.

Scenario 15: Memory is outdated

Use timestamps/TTL and update/delete rules.

Scenario 16: RAG agent retrieves wrong source

Evaluate retrieval independently and improve hybrid/reranking.

Scenario 17: SQL agent generates unsafe query

Read-only credentials + query restrictions + tenant filters.

Scenario 18: Coding agent deletes files

Sandbox + restricted filesystem + approval.

Scenario 19: Tool output is huge

Paginate/filter/summarize before adding to context.

Scenario 20: Context limit exceeded

Prune history and retrieve relevant state.

Scenario 21: Agent ignores required evidence

Reviewer + grounding requirement + evaluation.

Scenario 22: Tool unavailable

Detect failure, fallback or ask user; never claim success.

Scenario 23: Agent uses web instead of internal DB

Routing policy prioritizes authorized internal source.

Scenario 24: Ambiguous side-effect request

Ask clarification before execution.

Scenario 25: Agent sends message without approval

Separate draft and send permissions.

Scenario 26: Two sources conflict

Use authority/freshness policy and reviewer.

Scenario 27: Prompt injection succeeds

Adversarial tests + trusted policy layer.

Scenario 28: Behavior changes after model upgrade

Run regression suite and compare trajectories.

Scenario 29: Need reproducibility

Version model, prompt, graph, tools and state schema.

Scenario 30: Production incident caused by agent

Trace root cause and add a regression test.

PART 26 — CODING INTERVIEW QUESTIONS

Tool registry

```
class ToolRegistry:
    def __init__(self):
        self.tools = {}

    def register(self, name, fn):
        self.tools[name] = fn

    def execute(self, name, args):
        if name not in self.tools:
```

```
        raise ValueError("Unknown tool")
    return self.tools[name](**args)
```

Bounded loop

```
def bounded_agent(model, tools, state, max_steps=5):
    for _ in range(max_steps):
        action = model(state, tools=tools)

        if action["type"] == "final":
            return action["answer"]

        if action["type"] == "tool":
            args = validate(action["name"], action["args"])
            result = tools[action["name"]](**args)
            state["observations"].append(result)

    return "Stopped safely"
```

Tool-call deduplication

```
def call_key(name, args):
    return (name, tuple(sorted(args.items()))))

seen = set()
key = call_key("search", {"q": "BM25"})

if key not in seen:
    seen.add(key)
    # execute
```

Simple router

```
def route(query):
    q = query.lower()
    if "database" in q or "sql" in q:
        return "sql"
    if "policy" in q or "document" in q:
        return "retrieval"
    return "general"
```

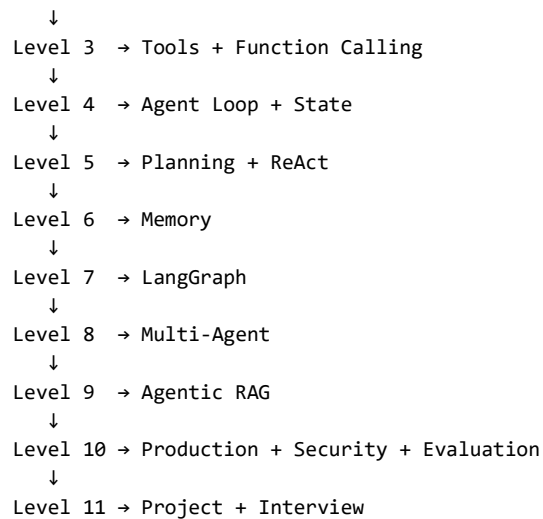
PART 27 — MATH / CS YOU ACTUALLY NEED

Concept	Why it matters
Graphs/state machines	Agent workflows and LangGraph
Hash maps	Tool registries, caches and deduplication
Queues	Task scheduling
Concurrency	Parallel independent tools
Distributed systems	Retries, idempotency and failures
Databases	Persistent memory/tool backends
Probability	Uncertainty and model outputs
Security	Authorization and least privilege

For basic agent interviews, systems thinking is more important than advanced mathematics. Learn state machines, APIs, databases, concurrency, failures, security and evaluation.

PART 28 — FINAL LEARNING ROADMAP

```
Level 1 → LLM Fundamentals
↓
Level 2 → AI Agents
```

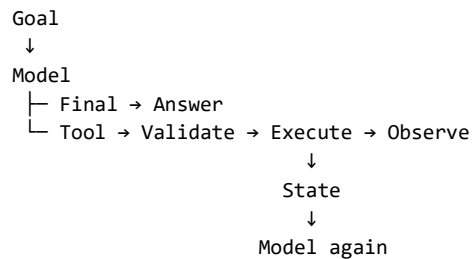



Level	Learn	Practice	Build
1	LLM/prompt basics	Model API + structured output	Chatbot
2	Agent concept	Manual loop	Weather/calculator agent
3	Tools/function calling	Schemas + validation	Multi-tool assistant
4	State/loops	Tracing + limits	Research agent
5	Planning/ReAct	Multi-step tasks	Travel/research agent
6	Memory	Short/long-term store	Personal assistant
7	LangGraph	Nodes/edges/checkpoints	Stateful agent
8	Multi-agent	Supervisor/parallel/reviewer	Research team
9	Agentic RAG	Iterative retrieval	Private research agent
10	Production	Security/evaluation	Deployed bounded agent
11	Interview	Design/coding/scenarios	Whiteboard project

PART 29 — CHEAT SHEETS

Agent architecture

AGENT = MODEL + TOOLS + STATE + CONTROL LOOP + POLICIES



CONTROL:

- ✓ permissions
- ✓ max steps
- ✓ timeout
- ✓ cost budget
- ✓ schemas
- ✓ idempotency
- ✓ approval
- ✓ tracing

Agent vs RAG vs Agentic RAG

	LLM	RAG	Agentic RAG
Flow	Generate	Retrieve → generate	Decide → retrieve/tools → observe → repeat
Actions	No	Usually no	Yes
Loop	No	Usually no	Yes
Complexity	Low	Medium	High
Use	Text	Document QA	Multi-step research

Interview design template

- 1. Clarify goal
- 2. Workflow or agent?
- 3. Define state
- 4. Define tools + schemas
- 5. Add retrieval if needed
- 6. Add graph/control loop
- 7. Add auth + validation
- 8. Add step/time/cost limits
- 9. Add tracing + evaluation
- 10. Add approval for risky actions

PART 30 — A–Z GLOSSARY

Agent

System that selects actions/tools and iterates toward a goal.

Agentic AI

AI built around dynamic decision-making and action.

Agentic RAG

Agent-controlled, iterative retrieval.

Checkpoint

Persisted execution state.

Control Loop

Repeated decision → action → observation → state cycle.

Function Calling

Structured model request for a declared function/tool.

Guardrail

Trusted behavioral or policy constraint.

Human-in-the-loop

Human review or approval inside execution.

Idempotency

Safe repetition without unintended duplicate effects.

LangGraph

Graph/state orchestration framework for agents.

Least Privilege

Minimum permissions required.

Memory

Persisted information retrievable into future context.

Memory Poisoning

Incorrect/malicious information enters memory.

Multi-Agent

Multiple agents collaborating.

Node

Unit of work in a graph.

Observation

Result of a tool/action.

Planning

Selecting actions to achieve a goal.

Prompt Injection

Untrusted content tries to change agent behavior.

ReAct

Action-observation style for tool use.

Reflection

Reviewing a result and improving it.

Replanning

Updating a plan after observations.

Router

Selects a tool/agent/workflow.

Sandbox

Restricted execution environment.

State

Current structured execution information.

Structured Output

Schema-constrained model response.

Supervisor

Coordinator that delegates to specialists.

Tool

Executable capability exposed to an agent.

Tool Routing

Choosing relevant tools for a task.

Tool Schema

Structured definition of inputs/outputs.

Trajectory

Sequence of decisions, actions and observations.

Workflow

Predetermined sequence of program steps.

PART 31 — FINAL INTERVIEW MASTER CHECKLIST

- Explain an AI agent in 30 seconds and 5 minutes.
- Draw the agent loop from memory.
- Explain why the model does not execute tools.
- Design safe function calling.
- Explain state vs context vs memory.
- Explain planning and ReAct.
- Compare workflow and agent.
- Design a supervisor multi-agent system.
- Explain LangGraph state/nodes/edges.
- Design Agentic RAG.
- Design step/time/cost limits.
- Explain idempotency.
- Design human approval.
- Explain prompt injection.
- Design memory safely.
- Evaluate trajectories and task success.
- Debug an agent from traces.
- Implement a basic agent loop in Python.
- Explain when NOT to use agents.
- Explain your project end-to-end with trade-offs.

60-second interview answer

An AI agent is a system around a model that can decide actions, call tools, observe results, maintain state and iterate toward a goal. Unlike a normal LLM call it can act; unlike basic RAG it can dynamically decide when and how to retrieve or use other tools. In production I would use strict schemas, trusted authorization, least privilege, bounded steps, timeouts, cost limits, checkpoints, tracing and human approval for high-impact actions. For complex workflows I can model the control flow as a state graph such as LangGraph. Agentic RAG combines this loop with retrieval so the agent can perform iterative, source-aware research.

Final principle

The goal is not maximum autonomy. The goal is useful, bounded autonomy: give the model enough freedom to solve uncertain parts while keeping execution, authorization, safety and business rules under reliable software control.

Scope

This guide covers the requested Agents & Agentic AI topics: AI agents, Agent vs LLM vs RAG, tools, function/tool calling, agent loops, planning, memory, ReAct, multi-agent systems, LangGraph and Agentic RAG, expanded into practical architecture, coding, production, security and interview preparation.

Q1. What is an AI Agent?

Answer:

An AI agent is a system that uses an LLM to understand a goal, reason about the required steps, select and use tools, observe their results, and iteratively take actions until the goal is completed.

Q2. Difference between LLM, RAG and Agent?

Answer:

An LLM primarily generates and understands language. RAG augments the LLM with external information through retrieval. An agent goes further by deciding what actions to take and using tools to accomplish a goal.

Q3. What is tool calling?

Tool calling allows an LLM to request execution of external functions or APIs using structured arguments. The application executes the tool and sends the result back to the LLM.

Q4. Does an LLM directly execute tools?

Usually no. The LLM generates a structured tool-call request, while the application or agent framework executes the actual function and returns the result.

Q5. What is the agent loop?

Goal

↓

Reason

↓

Action

↓

Observation

↓

Reason

↓

Action

↓

...

↓

Final Answer

Q6. What is ReAct?

ReAct stands for Reasoning and Acting. It combines reasoning with tool actions and observations in an iterative loop so the agent can decide what to do based on the results of previous actions.

Q7. What is agent memory?

Agent memory allows an agent to retain relevant information about the current task, conversation, or previous interactions so it can use that information later.

Q8. Short-term vs long-term memory?

Short-term memory maintains information within the current task or conversation, while long-term memory persists useful information across sessions and can be stored externally.

Q9. What is multi-agent AI?

A multi-agent system uses multiple specialized agents that collaborate or coordinate to solve a complex task.

Q10. Why use multiple agents?

To divide complex tasks into specialized responsibilities, improve modularity, and potentially execute independent tasks in parallel.

Q11. What is LangGraph?

LangGraph is a framework for building stateful, multi-step, controllable agent workflows using graphs consisting of nodes, edges, state, and conditional routing.

Q12. Why LangGraph instead of a simple chain?

A simple chain is mostly linear, while LangGraph supports loops, branching, conditional execution, shared state, human-in-the-loop workflows, and complex agent behavior.

Q13. What is Agentic RAG?

Agentic RAG is a RAG system where an agent dynamically decides when to retrieve information, which retrieval/tool to use, and whether additional retrieval is required before generating the final answer.

Q14. Normal RAG vs Agentic RAG?

Normal RAG:

Question → Retrieve → LLM → Answer

Agentic RAG:

Question → Agent

↓

Decide what to do

↓

RAG / Search / SQL

↓

Observe

↓

Decide again

↓

Answer

35. The most important mental model 🧠

Don't memorize 20 definitions separately.

Remember this:

LLM

|

"I can think/reason"

|

↓

AGENT

|

"I can decide"

|

↓

TOOLS

|

"I can do things"

|

↓

OBSERVATION

|

"I can see results"

|

↓

AGENT

|

"What next?"

|

↓

REPEAT

And:

RAG → gives knowledge

Tools → give capabilities

Memory → gives continuity

Planning → gives structured steps

Agent → decides what to do

ReAct → reason + act + observe

Multi-Agent → divides work

LangGraph → manages complex agent workflows

Agentic RAG → agent controls retrieval